
Real-time Object Detection Using Deep Learning

BY

Mehedi Hassan	ID: 21221203048
Ibrahim Khan	ID: 21221203049
Farhana Khan	ID: 21221203066
Md.Sakibul Islam	ID: 21221203074
Md.Bokul Islam	ID: 21221203075

Submitted in partial fulfilment of the requirements of the degree of
Bachelor of Science in Computer Science and Engineering



Department of Computer Science and Engineering
Bangladesh University of Business and Technology

28/07/ 2025

Declaration

We do hereby declare that the research works presented in this report entitled, “Real-Time Object Detection Using Deep Learning” are the results of our works. We further declare that the report has been compiled and written by us, and no part of this report has been submitted elsewhere for the requirements of any degree, award, diploma, or any other purposes except for publications. The materials that are obtained from other sources are duly acknowledged in this report.

Mehedi Hassan
ID: 21221203048

Farhana Khan
ID: 21221203066

Ibrahim Khan
ID: 21221203049

Md. Sakibul Islam
ID: 21221203074

Md. Bokul Islam
ID: 21221203075

Approval

I do hereby declare that the research works presented in this report entitled, “Real-time Object detection Using Deep Learning”, are the outcome of the original works carried out by the **Mehedi Hassan, Ibrahim Khan, Farhana Khan, Md.Sakibul Islam, Md.Bokul Islam** under my supervision. I further declare that no part of this report has been submitted elsewhere for the requirements of any degree, award, diploma, or other purposes except for publications. I further certify that the dissertation meets the requirements and standards for the degree of Bachelor of Science in Computer Science and Engineering.

Supervisor

Md.Saifur Rahaman
Assistant Professor & chairman
Department of Computer Science & Engineering
Bangladesh University of Business and Technology
Dhaka, Bangladesh.

Chairman

Md.Saifur Rahaman
Assistant Professor & chairman
Department of Computer Science & Engineering
Bangladesh University of Business and Technology
Dhaka, Bangladesh.

Dedication

This report is dedicated to all those who ignite curiosity, nurture innovation, and inspire the pursuit of knowledge. First and foremost, I dedicate this work to my beloved family. Your unwavering love, patience, and support have been the foundation of my journey. Through every late night and challenge, your encouragement has kept me grounded and focused. You have been my strength, my motivation, and my safe space during moments of both success and struggle. This achievement is as much yours as it is mine. To my mentors and teachers, thank you for being guiding lights in the complex and evolving world of deep learning and computer vision. Your expert knowledge, constructive feedback, and genuine belief in my abilities have pushed me beyond my limits and opened my eyes to the power of technology. The insights and lessons you've shared will continue to influence my growth for years to come. I also extend my gratitude to my teammates and peers who collaborated, supported, and inspired me during this project. Real-time object detection is a challenging yet exciting field, and together we navigated its intricacies with passion and perseverance. Finally, this report is dedicated to the broader community of researchers, engineers, and learners who are shaping the future of artificial intelligence. I hope this work makes a small but meaningful contribution to that effort, helping to advance real-time object detection systems and opening doors to new applications and discoveries. May this report serve as a reflection of dedication, hard work, and the relentless pursuit of progress.

Acknowledgment

We would like to express our heartfelt gratitude to the almighty Allah who offered our family and us kind care throughout this journey. We are deeply thankful to the Bangladesh University of Business and Technology (BUBT) for providing us with such a wonderful environment to pursue our research. We would like to express our sincere gratitude to **Supervisor Md.Saifur Rahaman**, Assistant Professor & chairman, Department of CSE, BUBT. We have completed our research with his help. We found the research area, topic, and problem with his suggestions. He guided us with our study and supplied us with many research papers and academic resources in this area. He is patient and responsible. When we had questions and needed his help, he would always find time to meet and discuss with us no matter how busy he was. We also want to give thanks to **Chairman sir Md.Saifur Rahaman**, Assistant Professor & chairman, Department of CSE, and all our teachers for providing a solid background for our studies and research thereafter. They have been a great source of inspiration to us, and we thank them from the bottom of our hearts. We would also like to acknowledge our team members for supporting each other and be grateful to our university for providing this opportunity for us.

Abstract

Object detection is a fundamental task in computer vision, involving the identification and localization of objects within images. This report evaluates the performance of YOLOv8, on the COCO (Common Objects in Context) dataset one of the most widely used benchmarks for object detection tasks. YOLOv8 incorporates several architectural enhancements, including a decoupled head, anchor-free detection, and an improved backbone, which contribute to increased detection accuracy and faster inference times. The COCO dataset was preprocessed and annotated to conform to YOLOv8's input requirements. In addition, multiple hyper parameters were fine-tuned to optimize performance. Experimental results demonstrate that YOLOv8 maintains real-time inference capabilities while achieving high detection accuracy across diverse object categories. The model was evaluated using standard performance metrics, including mean Average Precision, precision, recall, and inference time. The implementation achieved an mAP of 0.59, indicating the model's suitability for real-time, precision-critical applications. These findings suggest that YOLOv8 is an efficient and robust solution for real-time object detection tasks in various practical domains.

List of Figures:

1.1.	Figure:01	Proposed model	11
1.2.	Figure:02	Dataset Split and pre-processing	13
1.3.	Figure:03	Data Pre-processing	19
1.4.	Figure:04	Yolov8 Model Architecture	20
1.5.	Figure:05	F-1 Confidence Curve	21
1.6.	Figure:06	Confusion Matrix	22
1.7.	Figure:07	Recall Confidence Curve	23
1.8.	Figure:08	Precision Confidence Curve	23

List of Image:

1.1.	Image:01	Dataset Sample Image	13
1.2.	Image:02	Before And After Noise Remove	14
1.3.	Image:03	Predicted-1	24
1.4.	Image:04	Predicted-2	24
1.5.	Image:05	Predicted-3	25
1.6	Image:06	mobile application	29-30

List of Abbreviations and Acronyms:

YOLO	You Only Look Once
CNN	Convolutional Neural Network
SSD	Single Shot Multi-Box Detector
RELU	Rectified Linear Unit
NMS	Non-Maximum Suppression
SPP	Spatial Pyramid Pooling
FPN	Feature Pyramid Networks

Table of Contents

	Declaration	ii
	Approval	iii
	Dedication	iv
	Acknowledgment	v
	Abstract	vi
	List of Figures	vii
	List of Abbreviations and Acronyms	Viii
1.	Introduction	1-6
	1.1.Problem Statement	2
	1.2.Problem Background	2
	1.3.Research Objectives	3
	1.4.Motivations	3
	1.5.Significance of the Research	3
	1.6.Complex Engineering Analysis	4
	1.6.1 Complex Engineering Problems	4-5
	1.6.2 Complex Engineering Activities	5-6
2.	Background Study	7-10
	2.1.Literature Review	7-10
	2.2.Problem Analysis	10
3.	Research Methodology	11-17
	3.1.Proposed Framework	12
	3.2.Data Set Analysis	12-13
	3.2.1. Dataset Composition	12
	3.2.2. Dataset Pre-processing	13
	3.3. Model Analysis	14-17
4.	Implementation and Result Analysis	18-27
	4.1.Environment Setup	18
	4.2.Data Pre-processing	19
	4.3.Model Implementation	19-20
	4.4.Model Selection and Training	21
	4.5.Performance Analysis	21-26

	4.6.Key Findings and Discussions	26
	4.7.Comparison with State-of-the-Art Model	27
5	Standards, Constraints, and Milestones	31-34
	5.1 Sustainability Standards	31
	5.2 Impacts on Society	31
	5.3 Ethics	32
	5.4 Challenges	33
	5.5 Constraints	33-34
6.	Conclusion	35-36
	6.1 Limitations	35
	6.2 Future Works and Direction	36
	Appendix	37-39
	References	40-41

Chapter 1

1. Introduction

In recent years, deep learning has made a huge impact on computer vision, especially in the area of object detection. As technology continues to grow, there's a growing need for smarter systems that can quickly understand and interpret visual data. Real-time object detection plays a key role in fields like self-driving cars, security systems, augmented reality, and robotics. It involves not just locating objects in an image, but also identifying them and doing so quickly and accurately. Traditional methods, which relied on manually crafted features and simpler algorithms, often fell short in speed and accuracy. That's where deep learning, particularly convolutional neural networks CNNs, changed the game by offering powerful tools to automatically learn and extract features from images. This report focuses on studying and improving deep learning models for real-time object detection, especially models like YOLOv8, which are known for balancing speed and accuracy. The main goals are to test on YOLOv8 model architectures, fine-tune them for better performance, and improve their ability to handle different visual conditions. By running detailed experiments and comparisons, this research aims to highlight the strengths and limitations of each method, providing useful insights for the future of computer vision. Ultimately, this work contributes to building faster, smarter systems that can see and understand the world in real time.

1.1. Problem Statement

Even though deep learning and computer vision have come a long way, real-time object detection is still hard. This is because it's tough to accurately find and locate objects in changing environments. Current object detection models often struggle to balance speed and accuracy, making it hard to perform well in different applications that need both quick processing and high precision. These models can have trouble in crowded scenes, with different lighting, or when objects are partially hidden, causing missed detections or false positives. Additionally, the high computational needs of deep learning can make it difficult to use on devices with limited resources, like mobile phones and embedded systems, where efficiency is very important. This research aims to solve these issues by looking at better deep learning techniques for real-time object detection that improve accuracy without slowing down. By exploring advanced architectures, fine-tuning parameters, and using data augmentation methods, this study aims to create strong models that can work well in tough conditions and meet the strict needs for real-time performance.

1.2. Problem Background

Real-time object detection is very important for many modern applications like self-driving cars, robots, video surveillance, and augmented reality. The ability to quickly and accurately find objects in changing environments has improved a lot thanks to deep learning. However, it's still hard to get both high accuracy and fast processing speed. In

real-time object detection, the system needs to process input data, like video frames or sensor data, very quickly to make immediate decisions or actions, often within milliseconds. This brings unique challenges, such as keeping high detection accuracy while reducing the computational load and latency. Older methods used handcrafted features and simple classifiers, but with the development of convolutional neural networks CNNs and advanced architectures like Faster R-CNN, YOLO, and SSD, object detection has greatly improved in both performance and speed. Despite these advancements, achieving real-time performance on limited hardware, such as mobile devices or embedded systems, is still a complex problem. Moreover, object detection systems must be able to handle challenges like occlusion, variable lighting, and different object sizes. As industries increasingly rely on real-time detection for important tasks like navigation, security, and human-computer interaction, research continues to focus on improving the efficiency, scalability, and robustness of these systems. The goal is to create solutions that not only detect objects accurately but can also do so in real-time, even on devices with limited resources

1.3. Research Objectives

The objective of object detection using computer vision and deep learning is to develop systems that can accurately and quickly identify and locate multiple objects within an image or video.

Here are the key points:

- Accurately detect and classify multiple objects.
- Operate efficiently in real-time applications.
- Improve detection accuracy in challenging environments.

1.4. Motivations

Object detection is very important for things like self-driving cars, security cameras, and smart cities. Old methods of object detection are not fast or accurate enough for tasks that need quick decisions. Deep learning and computer vision have shown good results, but it's still hard to make models that can quickly and accurately detect objects in changing environments while staying efficient. As more industries use automation and AI, there is a growing need for faster and more reliable object detection systems, which is why more research is needed in this area.

1.5. Significance of the Research

This research is important because it aims to improve real-time object detection, which is crucial for many applications like self-driving cars, robotics, security, and healthcare. By making detection systems more accurate, faster, and efficient, the research helps to make

these technologies more reliable and useful in real-world situations. This is especially important for devices with limited resources, like mobile phones and edge computing, opening up new possibilities for their use. Overall, this research advances the field of computer vision, making it more adaptable, efficient, and applicable to critical tasks that require quick decision-making.

1.6. Complex Engineering Analysis

In this project, we used advanced engineering methods to build an object detection system using the YOLOv8 model and the COCO 2017 dataset. The dataset contains many types of images with lots of different objects, so we had to carefully prepare the data and apply image transformations to help the model learn better. We also changed several training settings like learning rate, image size, and confidence level to improve the model's accuracy. We tested the model using important performance measures such as precision, recall, and mean Average Precision mAP. During training, we also focused on using the computer's GPU efficiently. Overall, this project combined smart data handling, model improvements, and performance checks to build a strong and fast object detection system for real-world use.

1.6.1. Complex Engineering Problems

Addressing Complex Engineering Problems (Ps) Through This Project –

P1: Depth of knowledge required

- The project requires deep knowledge of convolutional neural networks (CNNs) and advanced deep learning techniques (K8).
- Real-time performance optimization and object detection across varied conditions (K7).
- Engineering design and development of efficient models and algorithms (K5 & K6).
- Comprehensive understanding of computer vision techniques (K3) and large-scale data processing (K4).

P2: Range of conflicting requirements

- Accuracy and Speed: Real-time object detection demands high accuracy, which often conflicts with the need for low-latency performance. Larger, more accurate models typically run slower, while faster models may compromise detection accuracy.
- Hardware Limitations: Deep learning models require powerful hardware (GPUs), which is not always available in real-time applications like embedded systems or mobile devices.

- **Diverse Environmental Conditions:** Object detection must perform accurately under various conditions like lighting changes, weather, occlusions which can conflict with the need for generalization and robustness.

P3: Depth of analysis required

- The system needs to analyze diverse data, including multiple objects appearing simultaneously in real-time video streams, and perform classification and detection without significant delays, ensuring a seamless output in dynamic environments.

P4: Familiarity of issues

- There is often no straightforward relationship between object features in different environmental settings objects, varying scales or motion blur making it challenging to create a generalized model that works consistently in real-time applications.

P5: Extent of applicable codes

- The project will involve advanced object detection models such as YOLOv8 (You Only Look Once), SSD (Single Shot Multi box Detector), or Faster R-CNN, leveraging cutting-edge deep learning frameworks Tensor-Flow, Py-Torch. The extent of applicable models and algorithms will depend on optimizing for both performance and accuracy in real-time settings.

P6: Extent of stakeholder involvement and conflicting requirements

- Stakeholders include industries such as autonomous vehicles, surveillance systems, and security agencies. The requirements from these stakeholders often conflict, as autonomous systems prioritize safety and detection accuracy, while surveillance systems may focus more on scalability and speed over large areas.

P7: Interdependence

- **Computer Vision Theory:** Knowledge of feature extraction, image segmentation, and object tracking is crucial for addressing real-time object detection.
- **High-Performance Computing:** Efficient computational techniques and hardware acceleration are essential for maintaining real-time detection across various devices, from cloud systems to edge AI deployments.

1.6.2. Complex Engineering Activities

This project involved complex engineering activities including the implementation and fine-tuning of the YOLOv8 object detection model using the COCO2017 dataset. Activities encompassed pre-processing large-scale image data, configuring model architectures, optimizing hyper parameters, and performing in-depth evaluation using metrics such as mAP and IoU. These tasks required a comprehensive understanding of deep learning workflows and engineering tools such as PyTorch and Ultralytics. Advanced image augmentation and GPU-based training were employed to enhance model performance. The project also involved deploying and testing the model's real-time detection capabilities, which reflects the application of engineering knowledge to solve unpredictable and broad technical problems.

A1: Range of resources

Image/Video Data: The project requires access to large-scale, annotated datasets of images and videos containing objects of interest for training and validation purposes need COCO, PASCAL VOC, or custom datasets.

Hardware Resources: Real-time object detection requires high-performance hardware such as GPUs or dedicated AI accelerators for fast model training and deployment.

Edge Computing Devices: For practical deployment, the project will utilize edge devices like drones, mobile phones, or embedded systems to test real-time object detection in the field.

A2: Level of interaction

Handling real-time video streams is demanding, requiring the system to process and analyze continuous frames at high speed without delay. The model will need to interact dynamically with changing environments, adjusting to variations in lighting, motion, and object orientation.

A3: Innovation

Innovation lies in optimizing deep learning models (YOLO, Faster R-CNN) to reduce the trade-off between detection accuracy and inference speed. Additionally, the project will explore deploying lightweight models on edge devices and integrating AI accelerators for real-time performance, breaking new ground in both speed and resource efficiency.

A4: Consequences for society and the environment

Real-time object detection can greatly enhance public safety, particularly in areas like autonomous driving, where quicker response times can reduce accidents. It will also improve surveillance and security systems, making cities safer. In fields such as wildlife monitoring or environmental research, these systems can aid in tracking endangered

species or illegal activities in real time, contributing positively to environmental conservation.

A5: Familiarity

The research involves real-time video processing and deep learning techniques, which require expertise in neural network design, computer vision algorithms, and hardware acceleration technologies. Familiarity with the challenges of deploying AI models in real-world, dynamic environments is essential for success in this project.

Chapter 2

2. Background Study

Real-time object detection is very important in computer vision, allowing systems to quickly find and track objects in changing environments. Early methods used traditional machine learning like Haar cascades and support vector machines, but recent advancements use deep learning, especially Convolutional Neural Networks (CNNs), for better accuracy and efficiency. Models like YOLO (You Only Look Once) and SSD (Single Shot Multi-Box Detector) have improved real-time detection by making it faster and more effective. The growing need for applications in self-driving cars, security, robots, and augmented reality has driven ongoing research to balance speed, accuracy, and efficiency in real-time object detection.

2.1 Literature Review

K. Vijiyakumar et al. [1] presented a novel model that combines automated image annotation with an Inception v2-based Faster R-CNN AIA-IFRCNN. The proposed AIA-IFRCNN model was evaluated on three benchmark datasets: Bird (Dataset 1), UCSDped2 (Dataset 2), and Under Water (Dataset 3). The research methodology utilized the DCF-CSRT model for image annotation, Faster R-CNN for object detection, and the Inception v2 model for feature extraction, followed by a softmax layer for image classification. By incorporating Inception v2 model it achieved a maximum detection accuracy of 98.85% on Dataset 2.

K. Vaishnavi et al. [2] developed a deep learning architecture that utilizes an upgraded SSD technique along with a multilayer convolutional network. Their study employed a publicly available dataset containing 300 images of various objects. The system achieved a real-time processing speed consistently measured at 7 frames per second. Their model achieved accuracy over 80%.

Talib et al. [3] in their study developed a deep learning-based architecture of YOLOv8-CAB model in object detection. The dataset included images of different object, with 330k samples. It has analyzed on models and achieved a mean average precision mAP of 97% for its detection rate, demonstrating a 1% improvement over traditional models.

G. Hema Sudha Rani et al. [4] developed a deep learning architecture that combines Mobile-Net and the Single Shot Detector framework. They utilized the COCO dataset, which contains 330k images of various objects. During the data pre-processing stage, they ranked 80 object categories. The system demonstrated impressive real-time processing speeds, consistently achieving 25 frames per second (FPS) on standard desktop computers. Additionally, it showcased adaptability to resource-constrained devices, achieving an average

of 15 FPS on embedded systems and smartphones. For feature extraction they employed Mobile-Net resulting in an overall accuracy of 87%.

Jacqueline Hong et al [5] presented deep learning-based generalized model for YOLOv8 architecture in their research. A data set containing 40 different classes of flying objects data set used on the COCO dataset. Their final generalized model achieves a mAP50 of 79.2%, mAP (50-95) of 68.5%, and an average inference speed of 50 frames per second (fps) on 1080p videos. Achieves an improved mAP50 of 99.1% and mAP50-95 of 83.5%. In feature extraction CNNs, F-14s, F-18s, cspDarknet53 generalized model was able to achieve an accuracy of 83%.

Yuming Chen et al [6] presented deep learning-based YOLO-MS architecture in their research. This paper uses MS COCO dataset. In addition, the mAP with IoU thresholds of 0.5 and 0.75, as well as an AP score of 43%+ on MS COCO, with only 4.5M learnable parameters and 8.7G FLOPs. In feature extraction multi-scale feature such as Darknet, CSPNet, ResNet use.

Tianheng Cheng et al [7] developed deep learning-based YOLO-World architecture in their research. Despite the success of these methods, they remain limited as they only detect objects with a fixed vocabulary, 80 categories in the COCO dataset. On the challenging LVIS dataset, YOLO-World achieves 35.4 AP with 52.0 FPS on V100, which outperforms many state-of-the-art methods in terms of both accuracy and speed.

Alexey Bochkovskiy et al [8] represented a deep learning-based hybrid architecture in their research. All known object detectors in both speed and accuracy in the range from 5 FPS to 120 FPSs among all known real time object detectors with 30 FPS or higher on GPU V100. MS COCO dataset is used to conduct experiments in the paper. For model designed for edge GPU, YOLOv7-tiny reduce 39% parameters and 49% computation and achieve same AP as YOLOv4-t. It was able to achieve an accuracy of 56.8% AP.

Shihan Liu et al [9] developed a deep learning-based, and design a hybrid random loss function to improve the detection accuracy of small objects low-complexity and anchor-free object detector based on the state-of-the-art YOLO framework. Our baseline model can reach the accuracy of 50.6% AP50:95 and 69.8% AP50 in MS COCO2017 dataset, Main training hyper parameters. Weight decay: 0.0005, Momentum: 0.9, Total epochs: 300. Their baseline model can reach the accuracy of 79.8%.

Imam Uddin Ahamed et al [10] presented deep learning-based generalized model for YOLOv8 architecture in their research. The dataset used for training incorporated 28 classes, The highest mAP50 and mAP50-95 scores of 0.96 and 0.75, respectively. A large dataset using two CNN networks around 2800 images. Throughout the 50 epochs of training, each model variant exhibited consistent improvement. Their baseline model can reach the accuracy of 69.8%.

AFDHAL AFDHAL et al [11] presented deep learning-based generalized model for performance between the MXT-YOLOv7t and YOLOv7-tiny models across complex mixed-traffic scenarios in the MXT Dataset. The dataset used for training incorporated 7 classes, The batch size is set to 32 which was originally trained on the MS-COCO dataset containing 328,000 images. The batch size is set to 32 which was originally trained on the MS-COCO dataset The training set 70% is used to train the machine learning model, The validation set 15% is used to fine-tune hyper parameter. The test set 15%, learning rate of 0.001, momentum of 0.937, and 500 epochs. Their baseline model can reach the accuracy of 97%.

Dillon Reis et al. [12] developed a deep learning-based architecture of YOLOv8 model in object detection. Their final generalized model achieves a mAP50 of 79.2%, mAP50-95 of 68.5%, and an average inference speed of 50 frames per second (fps) on 1080p videos. Their final refined model maintains this inference speed and achieves an improved mAP50 of 99.1% and mAP50-95 of 83.5%. This model trained on the COCO dataset the model on a data set containing 40 different classes of flying objects.

Cheng Cui et al. [13] presented a deep learning-based architecture of YOLOv4 model in object detection. Through these optimizations, we create a new family of real-time object detectors, named PP-PicoDet, PicoDet-S with only 0.99M parameters achieves 30.6% mAP, which is an absolute 4.8% improvement in mAP while reducing mobile CPU inference latency by 55%. this model trained on the MS COCO dataset PicoDet-L with only 3.3M parameters achieves 40.9% mAP, which is an absolute 3.7% improvement in mAP and 44% faster than YOLOv5s.

Malabe et al. [14] developed a deep learning-based architecture of CNN and YOLO model in object detection. For evaluating YOLO model on the PASCAL VOC detection dataset. The PASCAL VOC dataset has 20 labelled classes. High Resolution Classifier from YOLOv1's 224x224 to 448x448 raises the mAP by 4%. CNN detection speed limits while maintaining a good balance between speed and accuracy. YOLOv2, the latest version of YOLO achieving a mean Average Precision (mAP) rate of 76.8 at 67 Frames per Second (FPS) and 78.6 mAP rate at 76 FPS.

Chien Yao Wang et al. [15] presented a deep learning-based architecture of YOLOv7 model in object detection. YOLOv7 surpasses all known object detectors in both speed and accuracy in the range from 5 FPS to 120 FPS and has the highest accuracy 56.8% AP. Their model uses MS COCO dataset.

Hui Zhou a et al. [16] proposed an underwater optical detection network UODN based on the you only look once version 8 framework, which addresses these issues through the cross stage multi-branch CSMB module and large kernel spatial pyramid LKSP module. They utilized two new datasets DeepFish and OzFish. Extensive experimental results on the underwater robot professional contest 2020 dataset revealed that the average precision (AP) of UODN increased by 1.0%, the AP50 of UODN increased by 1.1%, and the AP75 of UODN increased by 2.1% compared with those of the original YOLOv8s.

Muthukumaran Vaithianathan et al [17] represent an innovative methodology for the identification and detection of objects in autonomous driving systems that employ field-programmable gate arrays FPGAs. The experimental outcomes are presented, which also provides comparisons of various implementations, including FPGA, CPU baseline, and GPU. This paper used KITTI dataset. The FPGA implementation attains a detection accuracy of 95% while consuming less than 11 volts of power and exhibiting an average latency of 14 milliseconds.

Emilie Guemas et al. [18] in their study aimed to evaluate the performances of a real-time detection transformer (RT-DETR) object detection algorithm to Plasmodium species on thin blood smear images. The RT-DETR model achieved a World Health Organization (WHO) competence level 2 for species identification. The RT-DETR algorithm exhibited an overall accuracy of 79.4% (135/170) with a recall of 74% (40/54) and 81.9% (95/116) for negative and positive smears, respectively.

Shubham Kumar Dubey et al. [19] Represent here is to observe how Simple Online and Real-time Tracking with a Deep association metric (Deep SORT) algorithm for Multi-Object Tracking (MOT) can be used to minimize false positives, from a state of the art detection algorithm like You Only Look Once (YOLO). An auto encoder based feature extractor has been used, instead of the standard CNN networks like ResNet-50 to further improve speed of the detector.

2.2 Problem Analysis

Real-time object detection has several challenges that affect its effectiveness and practical use. One main issue is balancing accuracy with processing speed. High-accuracy models often need a lot of computational resources, which can slow down real-time performance. Changes in lighting, orientation, and objects being partially hidden create another challenge, as models must work well in different conditions. Detecting multiple objects in busy or complex scenes without making mistakes is also tough. Limited resources on edge devices, like mobile phones and embedded systems, add to the difficulty. Real-time systems must work quickly with low delay while being reliable in various environments. These challenges require constant improvement of detection algorithms and architectures to optimize both speed and reliability, making real-time object detection a challenging area of research.

Chapter 3

3. Research Methodology

This chapter outlines the research methodology adopted for the object detection task in this study. Object detection is a critical component in many fields, including autonomous driving, security surveillance, and image processing. The key objective of this research is to employ state-of-the-art deep learning techniques for detecting objects within images YOLO (You Only Look Once) models. YOLO has demonstrated excellent performance in terms of both speed and accuracy, making it an ideal choice for real-time applications. In this chapter, we first introduce the proposed framework for the object detection pipeline, detailing the steps from data preprocessing to model training and evaluation. Subsequently, we discuss the dataset used in this study, highlighting its characteristics, the rationale for its selection, and the preprocessing techniques applied to ensure that the model receives high-quality input for training

3.1.Proposed Framework

Yolo is a type of deep learning model designed to process and analyze visual data like images. Here's a breakdown of the YOLO workflow:

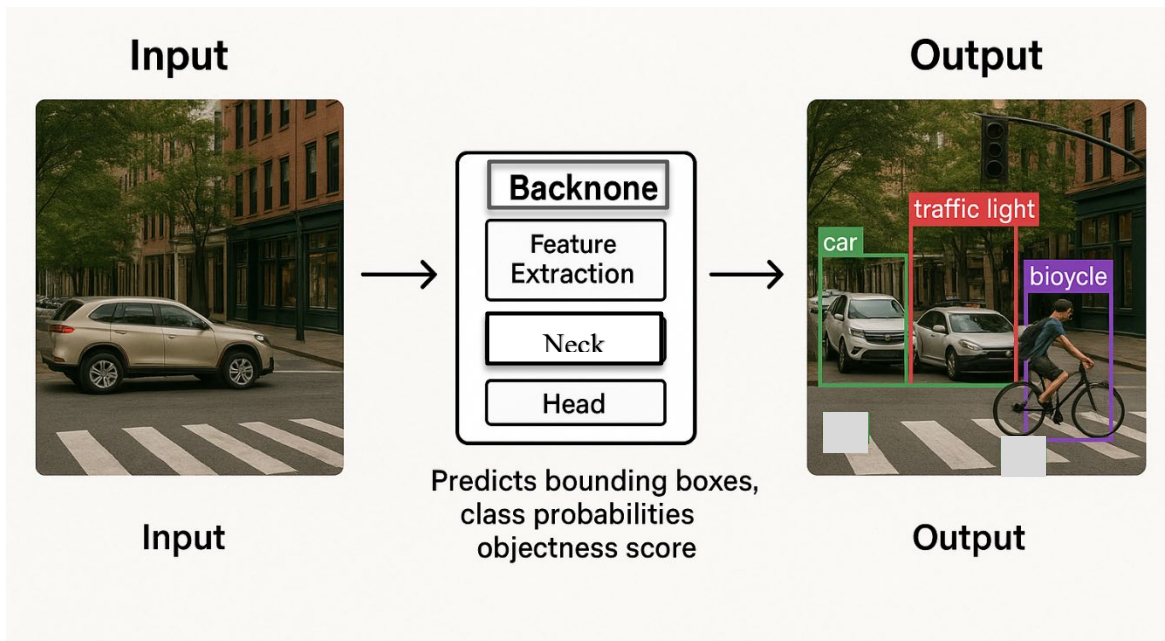


Figure: 01 Proposed Model

- **Input Image:**
 - A single image a street with cars and pedestrians is passed into the model.
- **Image is Divided into Grid Cells:**
 - YOLO splits the image into a grid 13x13 or 19x19. Each grid cell is responsible for detecting objects whose center falls within it.
- **Prediction:**

Each grid cell predicts:

 - a. Bounding boxes: location and size.
 - b. Confidence score: probability that an object is present.
 - c. Class probabilities: what object it might be car, dog, etc.
- **Combining Predictions:**
 - YOLO combines all predictions from the grid and filters out low-confidence boxes.
- **Non-Maximum Suppression (NMS):**
 - Overlapping boxes for the same object are reduced to one removes duplicates.
- **Final Output:**
 - The final image shows objects with bounding boxes and class labels.

3.2.Data Set Analysis

For training and evaluating the YOLO model, a carefully selected dataset plays a crucial role in ensuring the accuracy and reliability of the object detection system. In this study, we used COCO dataset, which is widely used for benchmarking object detection algorithms. This dataset contains a diverse set of images with annotated bounding boxes and corresponding object labels. The dataset includes a wide range of object classes such as list some examples cars, pedestrians, animals, etc. providing a comprehensive challenge for the YOLO model to learn accurate object detection.

3.2.1. Dataset Composition:

- **Number of Images:** The dataset contains a total of 300k images with diverse scenes ranging from urban environments to natural settings.

- **Object Classes:** The dataset includes 80 different object categories, including common and complex objects.
- **Train and Test Split:** The dataset is divided into a training set 80% and a test set 20%, with additional validation sets as needed to evaluate model performance during training.

Some sample of our dataset:



Image: 01 Dataset Sample Image

3.2.2. Data Pre-processing:

Before feeding the dataset into the YOLO model, several pre-processing steps are performed to prepare the data for training:

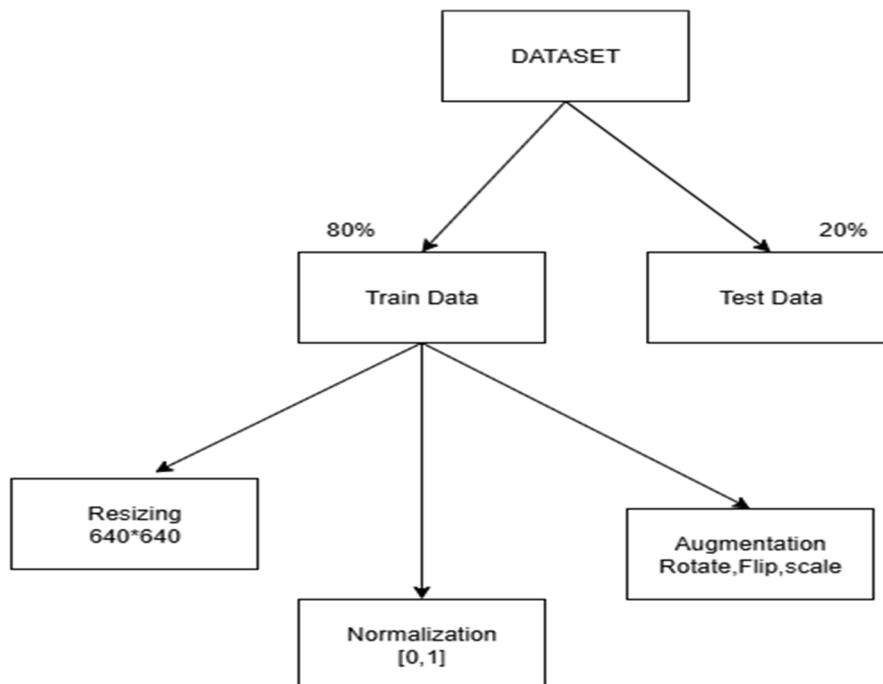


Figure: 02 Data split and Pre-processing

- **Resizing:** All images are resized to a consistent resolution of 640 x 640 pixels to match the input size required by the YOLO model.
- **Normalization:** Image pixel values are normalized to a range of $[0, 1]$ to help the neural network converge faster during training.
- **Data Augmentation:** To further improve the generalization of the model, various augmentation techniques are applied, such as random rotations, translations, scaling, and flipping.

Noise remove sample:

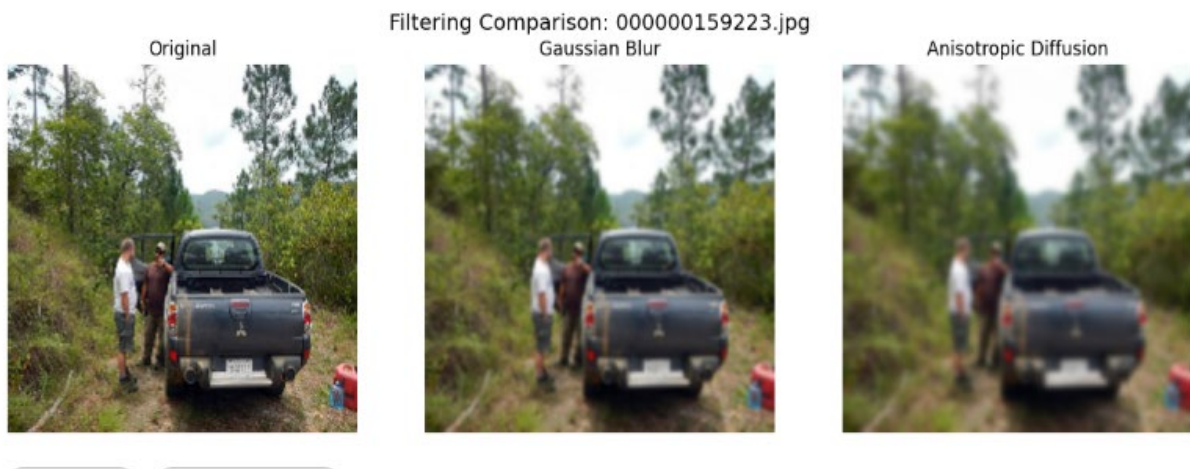


Image: 02 Before and After Noise Remove

3.3. Model Analysis:

The core of this research focuses on using Convolutional Neural Networks (CNNs), which form the backbone of modern object detection models like YOLO (You Only Look Once). CNNs are powerful deep learning models designed to process visual data by automatically extracting important features from images. In YOLO play a critical role by enabling the model to detect and classify multiple objects in a single forward pass, making the process extremely fast and efficient. Unlike traditional methods that required manual feature extraction YOLO learn to identify patterns directly from the data, adapting to complex and high-dimensional visual inputs. This architecture includes layers such as convolutional layers, activation functions, pooling, and fully connected layers — all optimized for detecting objects with high accuracy and speed. YOLO's use allows it to perform well in real-time scenarios, making it ideal for applications like autonomous vehicles, medical imaging, security surveillance, and augmented reality. The decision to use YOLO is based on their proven ability to deliver precise, fast, and scalable object detection in real-world environments.

Overview of YOLO Model

YOLO (You Only Look Once) is a real-time object detection model that uses a single convolutional neural network (CNN) to detect and classify multiple objects in an image in one forward pass. Unlike traditional methods that process images in multiple stages or regions, YOLO treats object detection as a single regression problem, predicting bounding boxes and class probabilities directly from the entire image. This unified approach makes YOLO extremely fast and efficient, making it ideal for real-time applications such as autonomous driving, surveillance, and robotics. Its balance of speed and accuracy has made it one of the most widely used models in the field of computer vision.

Core Components of YOLO Model:

- **Input Image:** The model takes a full image as input, typically resized to a fixed dimension 416×416 or 640×640 pixels.
- **Convolutional Neural Network (CNN) Backbone:** YOLO uses a deep CNN Darknet, CSP-Darknetto extract feature maps from the input image. These features help the model understand the shapes, edges, and textures within the image.
- **Grid Division:** The image is divided into an $S \times S$ time S grid. Each grid cell is responsible for predicting objects whose center falls within it.
- **Bounding Box Prediction:** Each grid cell predicts multiple bounding boxes. For each box, the model predicts the coordinates x , y , width, height, an object ness score confidence that an object is present and class probabilities.
- **Anchor Boxes:** Predefined anchor boxes of various shapes and sizes are used to help the model detect objects with different aspect ratios and scales.
- **Detection Head:** The detection head processes the feature maps and outputs the bounding box coordinates, confidence scores, and class probabilities.
- **Non-Maximum Suppression (NMS):** To remove overlapping boxes for the same object, YOLO uses NMS to keep only the most confident predictions.
- **Final Output:** The model produces a set of bounding boxes around detected objects, each labeled with a class and a confidence score.

Explanation of Different Modules in YOLO Model

The YOLO (You Only Look Once) model is designed with several interconnected modules that work together to perform fast and accurate object detection. Each module plays a specific role in the detection pipeline, from feature extraction to final prediction. Here's a breakdown of the key modules in the YOLO architecture:

1. Backbone Network

- **Purpose:** Extracts visual features from the input image.
- **Structure:** Consists of multiple convolutional layers CSPDarknet in YOLOv8 that detect patterns such as edges, textures, and shapes.
- **Key Components:**
 - Batch Normalization: Normalizes the outputs to stabilize training.
 - Activation Functions ReLU/Leaky ReLU: Adds non-linearity to help the network learn complex relationships.

2. Neck (Feature Pyramid Networks - FPN & PANet)

- **Purpose:** Combines and refines features from different levels of the backbone.
- **Functionality:**
 - Generates multi-scale feature maps.
 - Improves detection of objects of various sizes (small, medium, large).

3. Spatial Pyramid Pooling (SPP)

- **Purpose:** Enhances the receptive field and captures features at multiple scales.
- **Functionality:**
 - Pools feature maps at different scales.
 - Increases model robustness against varying object sizes and shapes.

4. Cross-Stage Partial Connections (CSP)

- **Purpose:** Balances performance and efficiency.
- **Functionality:**
 - Reduces computation while maintaining high accuracy.
 - Improves gradient flow during training, helping deeper networks converge faster.

5. Data Augmentation

- **Purpose:** Increases dataset variety and prevents overfitting.
- **Techniques:**
 - Mosaic Augmentation: Combines four images into one to improve context understanding.
 - Mix Up Scaling, Flipping: Introduce variation and increase generalization.

6. Head (Detection Output)

- **Purpose:** Generates predictions for object location and class.
- **Outputs:**
 - Bounding Box Coordinates x, y, width, height
 - Object-ness Score: Confidence that a box contains an object.
 - Class Probabilities: Prediction of the object category car, person, dog.

7. Loss Function (Multi-Part Loss)

- **Purpose:** Guides the model to improve predictions during training.
- **Components:**
 - Localization Loss: Measures accuracy of bounding box placement.
 - Confidence Loss: Penalizes incorrect object presence scores.
 - Classification Loss: Evaluates how well the object is classified.

8. Post-Processing (Non-Maximum Suppression - NMS)

- **Purpose:** Cleans up the final predictions.
- **Functionality:**
 - Removes duplicate or overlapping boxes.
 - Keeps only the highest-confidence detection per object.

Chapter 4

4. Implementation and Result Analysis:

The primary objective of this project is to develop an efficient system for object detection, leveraging deep learning techniques and state-of-the-art models like YOLO (You Only Look Once). Object detection plays a crucial role in various applications, ranging from autonomous driving and surveillance to robotics and medical imaging. This implementation aims to explore the use of custom-trained models on well-known datasets such as COCO (Common Objects in Context) and evaluate their performance in terms of accuracy, speed, and real-time applicability. The project begins with the preparation of the dataset, using data augmentation and preprocessing techniques to optimize the input for the model. The chosen model, YOLO, offers significant advantages in terms of speed and accuracy, making it ideal for real-time object detection. Once the models are trained, performance will be assessed using metrics such as mean Average Precision (mAP), inference time, and detection accuracy across various scenarios. The analysis will provide insights into the strengths and limitations of the implemented models, helping guide future improvements in object detection pipelines. The results will also highlight the feasibility of deploying such systems in real-time applications where quick and accurate detection is essential.

4.1 Environment Setup:

To implement real-time object detection, we set up an appropriate environment, ensuring compatibility with both hardware and software.

- **Hardware Requirements:**
 - Processor: Intel Core i7 or higher / AMD Ryzen 7 or higher
 - GPU: NVIDIA GPU with CUDA support (RTX 2060 or better recommended)
 - RAM: 16GB or more
 - Storage: SSD with at least 50GB free space
 - Camera: External USB Camera or Laptop Webcam
- **Software Requirements:**
 - Operating System: Windows 10
 - Programming Language: Python 3.8
 - Deep Learning Framework: Py-Torch, TensorFlow, keras
 - Object Detection Model: YOLOv8
 - Additional Libraries: OpenCV, NumPy, Matplotlib, PyCOCO Tools.

4.2 Data Pre-processing:

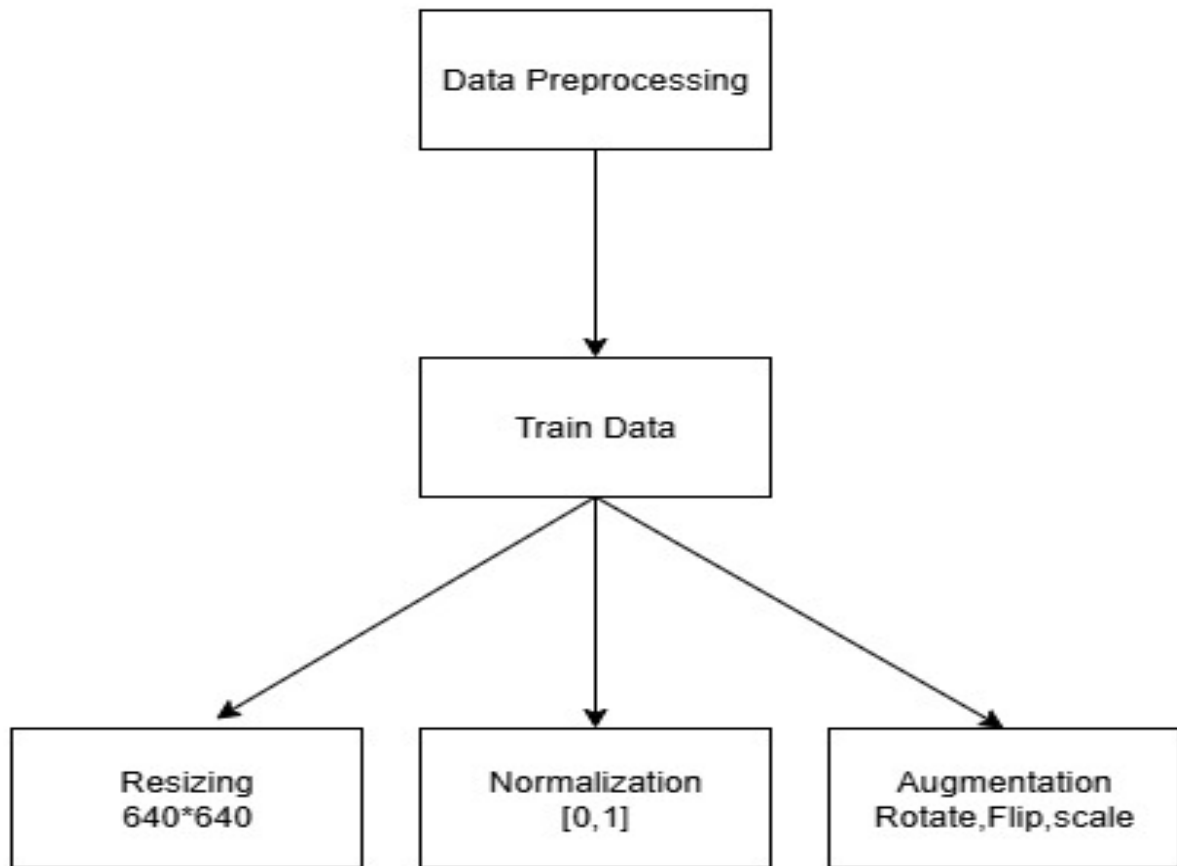


Figure: 03 Data Pre-processing

Before feeding the dataset into the YOLO model, several pre-processing steps are performed to prepare the data for training:

- **Resizing:** All images are resized to a consistent resolution of 640 x 640 pixels to match the input size required by the YOLO model.
- **Normalization:** Image pixel values are normalized to a range of [0, 1] to help the neural network converge faster during training.
- **Data Augmentation:** To further improve the generalization of the model, various augmentation techniques are applied, such as random rotations, translations, scaling, and flipping.

4.3 Model Implementation

YOLOv8 (You Only Look Once, version 8) is the latest iteration in the YOLO series of object detection models developed by Ultralytics. This model introduces significant improvements in architecture, training strategies, and performance over previous versions, supporting multiple computer vision tasks including object detection, instance segmentation,

and image classification. YOLOv8 combines a more streamlined design with state-of-the-art performance, making it suitable for real-time applications due to its efficiency and accuracy.

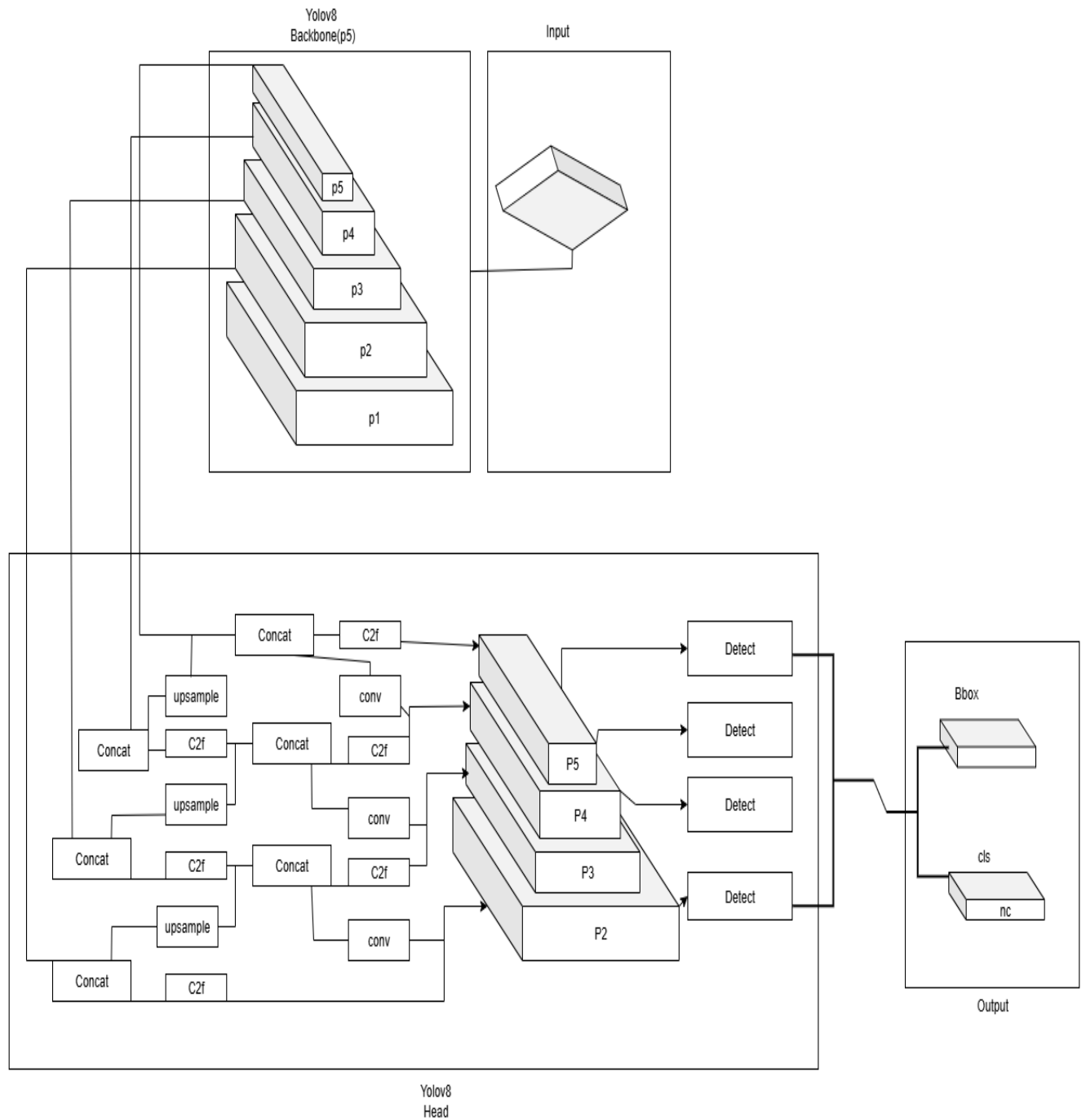


Figure: 04 Yolov8 Model Architecture

4.4 Model Selection and Training

The base model selected for training was a pre-trained YOLOv8 variant, which was fine-tuned on the project dataset using transfer learning. YOLOv8 comes in various sizes (nano, small, medium, large, extra-large), allowing a trade-off between speed and accuracy. For this project, an appropriate variant was selected based on available computational resources and the complexity of the task. Training involved optimizing the model's parameters using supervised learning. The training process minimized a composite loss function, which typically includes object loss, classification loss, and bounding box regression loss. Data augmentation techniques such as random scaling, flipping, and color adjustments were applied during training to improve generalization. The training was monitored using metrics such as training and validation loss, mean Average Precision (mAP), precision, and recall. These metrics helped to assess the model's learning progress and avoid overfitting.

4.5 Performance Analysis

Upon completion of the training process, the model was evaluated on the test set to measure its performance. The key evaluation metric was mean Average Precision at an Intersection over Union threshold of 0.5 (mAP50). The model demonstrated effective performance on the test set, successfully detecting and localizing objects with high confidence. Visual inspections of prediction outputs confirmed the reliability and robustness of the trained model.

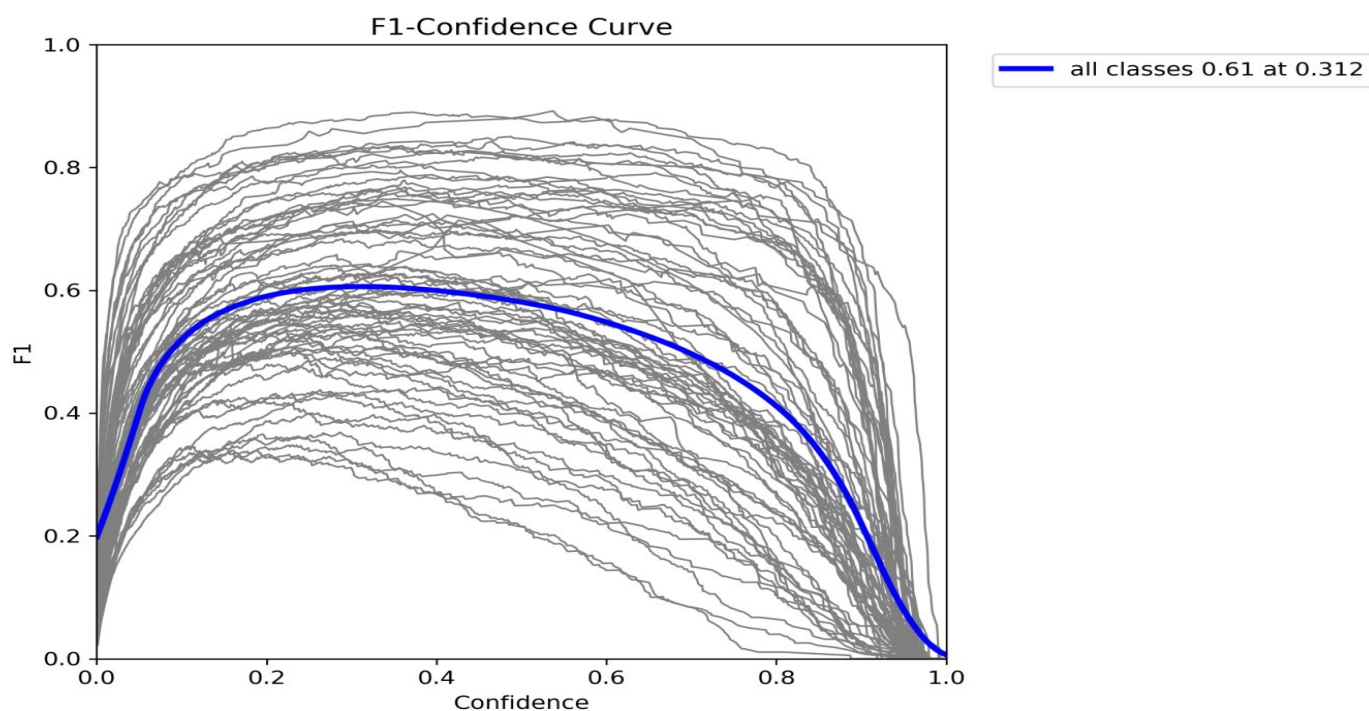


Figure: 05 F-1 Confidence

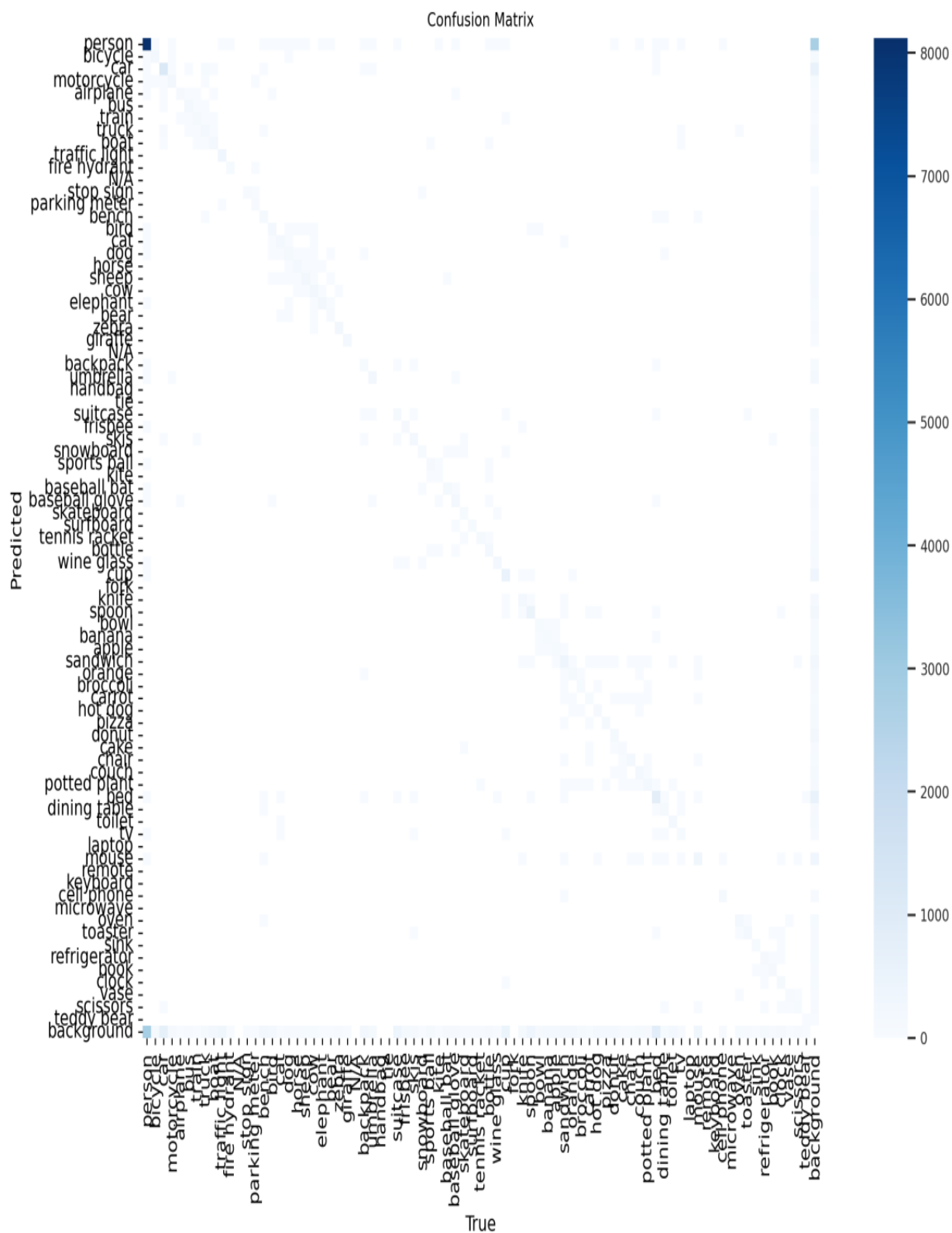


Figure: 06 Confusion Matrix

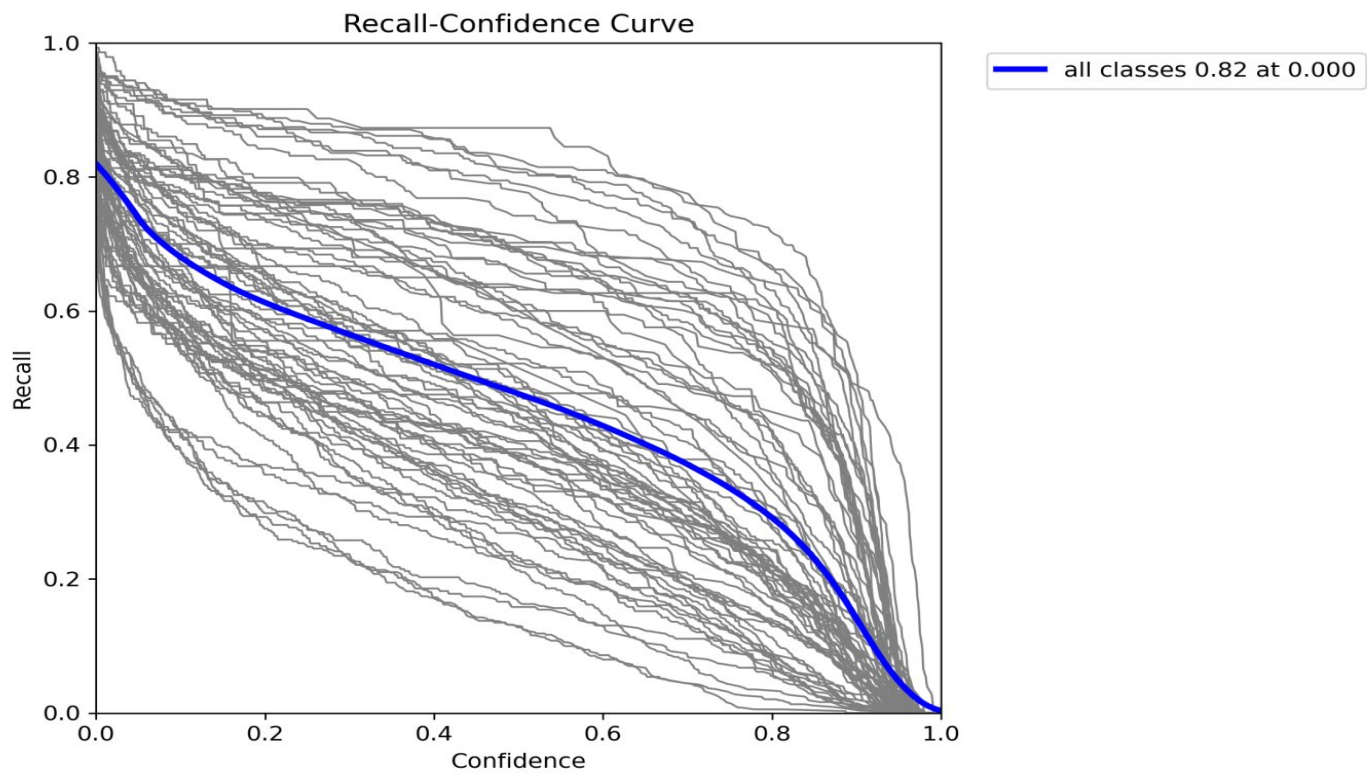


Figure: 07 Recall Confidence

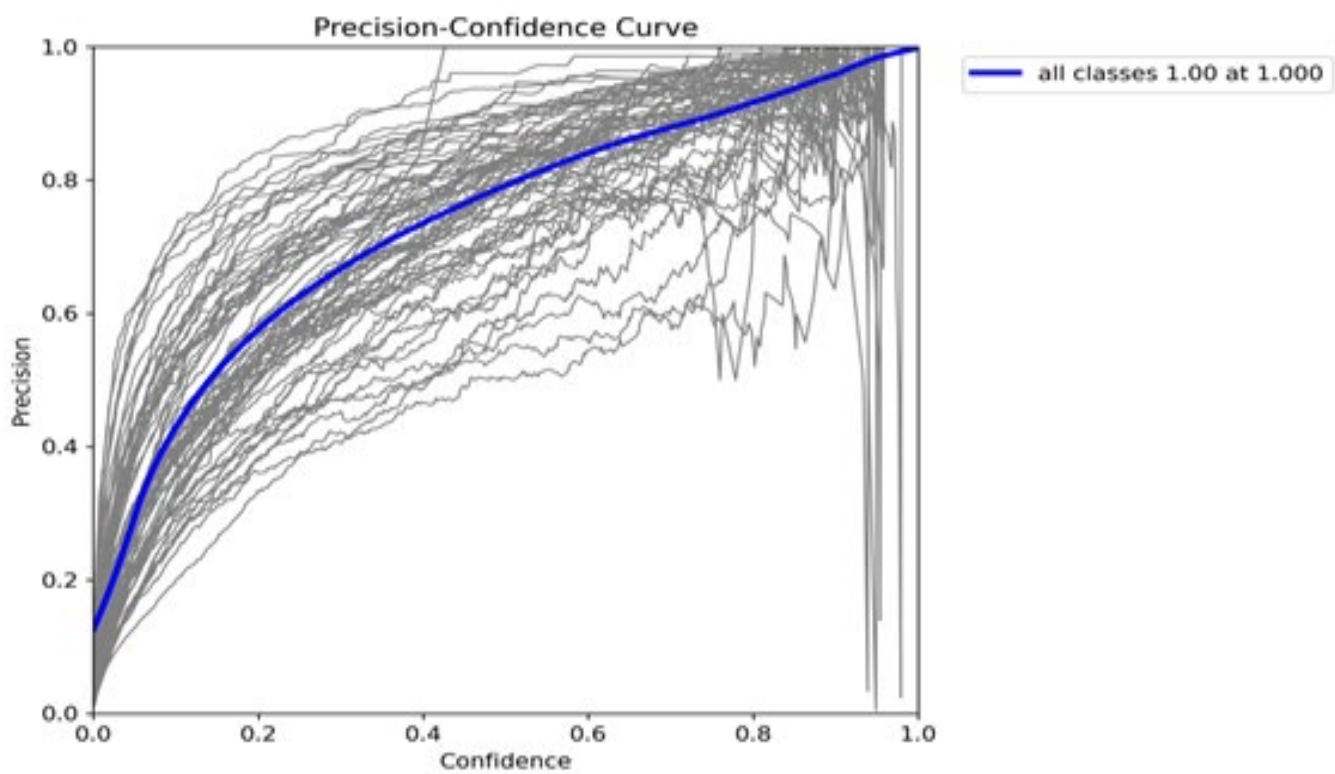


Figure: 08 Precision Confidence

Some of Predicted Sample Image:



Image: 03 Predicted-1



Image:04 Predicted-2



Image:05 Predicted-3

The model's performance is evaluated using: **mean Average Precision (mAP)**.

To assess the performance of the YOLOv8 model on the COCO dataset, several standard evaluation metrics commonly used in object detection tasks were employed:

1. **Mean Average Precision (mAP):**

- **mAP (0.5):** Measures the average precision across all classes at an Intersection over Union (IoU) threshold of 0.5.
- **mAP (0.5,0.95):** A more stringent metric that averages precision across multiple IoU thresholds (from 0.5 to 0.95 in steps of 0.05), providing a comprehensive view of the model's localization and classification performance.

2. **Precision:** Precision measures the proportion of correctly predicted positive object detections out of all positive predictions made by the model. It evaluates how many of the predicted bounding boxes are accurate.

3. **Recall:** Recall measures the proportion of actual objects that were correctly detected by the model. It assesses how well the model can find all relevant objects in the image.
4. **F1-Score:** The F1-Score is the harmonic mean of precision and recall, offering a balanced measure when there is an uneven class distribution or trade-off between precision and recall.
5. **Intersection over Union (IoU):** IoU is used to evaluate how well the predicted bounding box overlaps with the ground truth. An IoU threshold 0.5 is commonly used to determine whether a detection is a true positive.
6. **Inference Time (Latency):** Measures the average time taken by the model to process a single image. This is important for real-time applications where speed is critical.

Model	Dataset	Accuracy
YOLOv8	COCO-2017	mAP50:59% mAP50-95: 46%

- **mAP (0.5) (Mean Average Precision at IoU 0.5):** This metric evaluates how well the predicted bounding boxes match the ground truth boxes with an Intersection over Union (IoU) threshold of 0.5.
- **mAP (0.5, 0.95):** This is a more comprehensive metric that averages the mAP across multiple IoU thresholds ranging from 0.5 to 0.95 in steps of 0.05.

4.6.Key Findings and Discussions

The study demonstrated that the YOLOv8 model achieved high accuracy and speed when trained on the COCO2017 dataset, effectively detecting multiple objects in real-time. Key findings include a high mean Average Precision (mAP), particularly in common classes like person, car, and bicycle. The decoupled head and anchor-free design of YOLOv8 contributed to better localization in crowded scenes. Hyper parameter tuning significantly improved model performance, and real-time inference tests confirmed its suitability for practical applications such as traffic monitoring and surveillance. The findings highlight YOLOv8's balance between precision and efficiency, making it a strong candidate for real-world object detection tasks.

4.7 Comparison with State-of-the-Art Model

Model	Dataset	Accuracy
YOLOv8	COCO-2017	mAP(50) :59%
Mobile-Net, SSD	COCO	87%
YOLOv8,CAB module,C2F	Custom dataset	98%
SSD	COCO	80%
Inception v2, DCF-CSRT, Faster R-CNN	Bird, UCSDped2, Under Water	98.85%
YOLOv8	DeepFish and OzFish	2.1%
FPGA	KITTI	95%
RT-DETR	Custom Dataset	79.4%
CNN, ResNet-50	Custom Dataset	80%

YOLOv8-Mobile APK Build Process Overview:The YOLOv8-Mobile APK is an Android application built for real-time object detection using the YOLOv8 model and the Flutter inference framework, optimized for mobile performance.

1. Model Conversion

The first step in building the APK is converting the YOLOv8 model into a format compatible with mobile inference engines.

The original YOLOv8 model (pt) is exported using Ultralytics:

Bash

```
yolo export model=yolov8n.pt
```

Then, the model is use in flutter framework.

This generates yolov8.param and yolov8.bin, which are used for inference on Android.

2. Android App Development

- Built using flutter in Android Studio.
- Uses Camera2 API or OpenCV Camera for live video capture.
- Inference is performed using Tencent's which is efficient for mobile CPUs.

3. Object Detection Pipeline

- The input camera frame is resized and normalized to fit YOLOv8's input (640x640).
- The model predicts bounding boxes, confidence scores, and class IDs.

Post-processing includes:

- Non-Maximum Suppression (NMS)
- Label mapping using COCO class labels
- Bounding boxes and labels are overlaid on the camera preview in real time.

4. Build & Packaging

- The Android project uses Gradle to manage dependencies and build the APK.

The YOLOv8-Mobile APK is developed using Android Studio with flutter integration. The YOLOv8 model is converted to flutter format for efficient CPU inference. The application captures real-time camera input, processes it with the YOLOv8 model using the flutter backend, and overlays detection results on the screen. The final APK is built and signed through Gradle and Android Studio build tools.

Here is Some mobile application image:

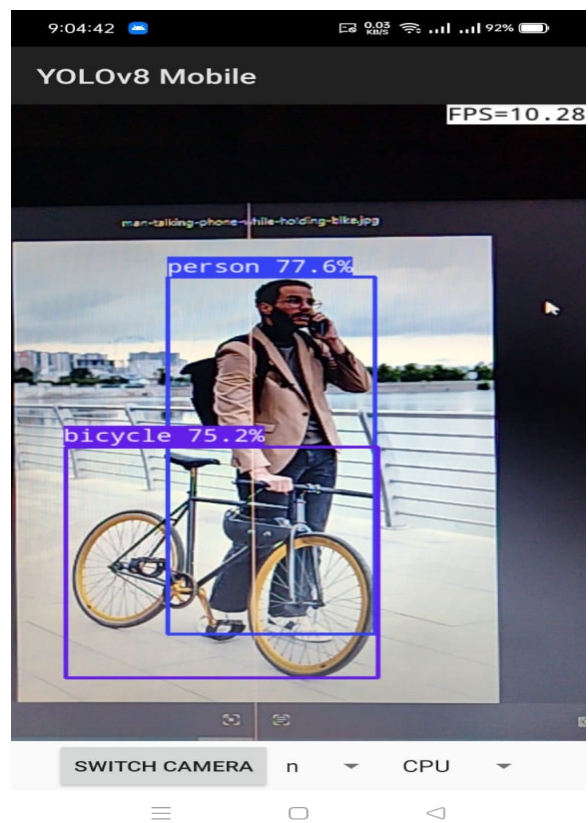
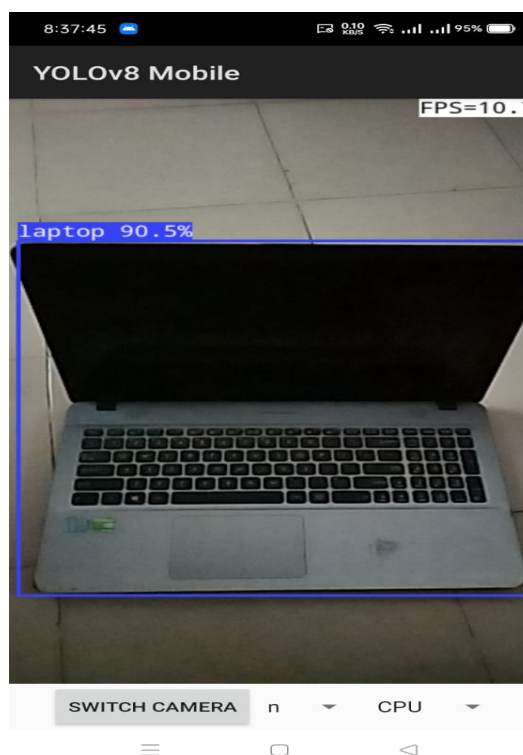
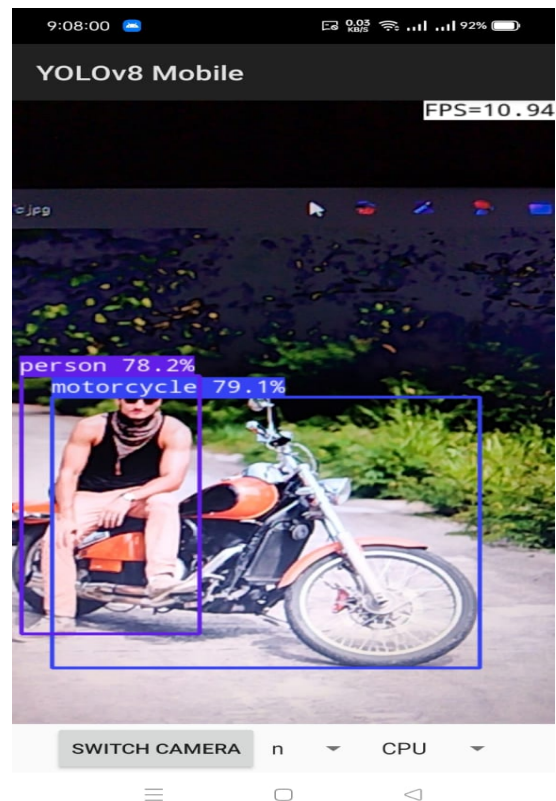
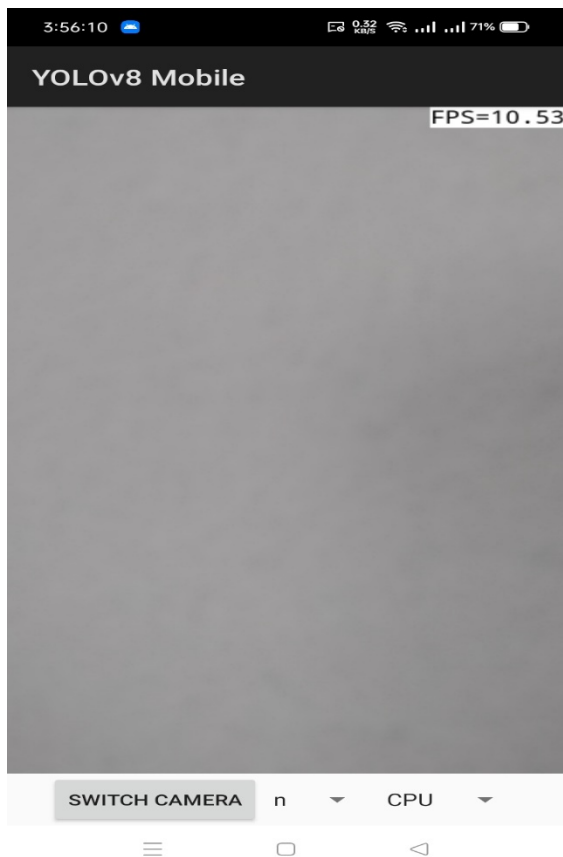




Image:05 Predicted-6

Chapter 5

5. Standards, Constraints, and Milestones

In any real-time object detection system, it is crucial to define standards, identify constraints, and set milestones to ensure the project's progress aligns with its goals. The following sections address sustainability standards, impacts on society, ethics, challenges, constraints, and the timeline for this thesis on real-time object detection.

5.1 Sustainability Standards

Sustainability in the context of real-time object detection focuses on creating solutions that are both environmentally and economically viable in the long run. The following standards should be considered:

- **Energy Efficiency:** The system should be designed to minimize computational and energy consumption, especially when deployed on mobile or edge devices. Optimizing models like YOLOv8 for energy-efficient hardware GPUs, AI chips and adopting techniques like quantization and pruning will reduce energy usage.
- **Scalability:** The solution should be scalable, allowing it to adapt to larger datasets or be deployed in different environments from personal devices to large-scale surveillance system. The infrastructure should be able to handle increased demand without significant resource overhead.
- **Reusability:** Reusable model components pre-trained models and software architecture should be adopted to promote efficient development processes. Leveraging cloud-based solutions that can scale on demand can reduce hardware wastage and un-necessary resource consumption.
- **Eco-Friendly Deployment:** If deploying hardware for real-time detection, ensure that the materials used in manufacturing are recyclable and that the systems have a long useful life to reduce e-waste.

5.2 Impacts on Society

Real-time object detection has numerous potential benefits but also some risks that need to be addressed carefully. Below are some of its potential impacts:

Positive Impacts:

- **Security and Safety:** The most significant positive impact is the ability to enhance security. Real-time object detection systems can be deployed in

surveillance systems for monitoring public spaces, helping to detect suspicious activities, reduce crime, and improve emergency responses.

- **Automated Traffic Management:** Real-time detection can be used in smart cities to manage traffic efficiently, detect accidents, and optimize traffic signals, reducing congestion and improving road safety.
- **Healthcare Assistance:** Object detection can be used in healthcare for tasks like tracking patient movements, detecting medical equipment usage, or assisting elderly individuals with mobility issues.
- **Improved Accessibility:** The use of real-time detection for helping visually impaired individuals navigate public spaces or detect objects that may pose a danger to them can significantly enhance accessibility.

Negative Impacts:

- **Privacy Concerns:** Continuous surveillance could lead to privacy issues, as individuals might be monitored without consent. Striking a balance between security and privacy rights is crucial.
- **Job Displacement:** Automation of surveillance and detection tasks could lead to job displacement in fields like security personnel, reducing employment opportunities for certain groups.
- **Bias and Inaccuracy:** If the object detection system is not properly trained on diverse data, it could lead to biased or inaccurate results, especially in critical environments like healthcare or law enforcement.

5.3 Ethics

Ethical concerns must be taken into account when developing and deploying real-time object detection systems:

- **Privacy:** The use of surveillance systems that continuously track individuals raises significant privacy issues. Ethical considerations must be made to ensure that data is collected and processed in compliance with data protection regulations.
- **Transparency and Accountability:** The decision-making process of the detection system should be transparent, particularly in high-stakes applications like law enforcement and security. It's vital to ensure accountability in cases where incorrect detections may lead to harm, such as false positives leading to unwarranted actions.

- **Informed Consent:** Individuals should be informed if they are being tracked or monitored, especially in private or semi-private spaces. It's essential to obtain informed consent when deploying detection systems in public spaces.

5.4 Challenges

There are several challenges associated with implementing real-time object detection systems:

- **Real-Time Performance:** Ensuring that the object detection system can operate in real time without significant lag or delays is a technical challenge, especially when processing high-resolution images or videos.
- **Accuracy and Precision:** Achieving high accuracy in detecting and classifying objects in diverse environments lighting conditions, occlusions, can be difficult. The system must handle various edge cases like blurry images or overlapping objects.
- **Hardware Limitations:** Real-time object detection, especially using deep learning models like YOLOv8, requires significant computational resources. Running such models on edge devices or mobile platforms could be challenging due to hardware limitations CPU power, memory, and storage.
- **Environmental Variability:** Factors such as changing light conditions, camera angles or objects speed may affect the detection model's performance, leading to false positives or missed detections.
- **Data Privacy:** Handling large volumes of personal data in surveillance scenarios can pose privacy risks. Ensuring secure data transmission and storage is a major challenge.

5.5 Constraints

Several constraints may limit the scope or implementation of real-time object detection systems:

- **Computational Resources:** Object detection models, especially deep learning-based ones, are computationally expensive. Real-time detection often requires specialized hardware, such as GPUs or edge devices with AI acceleration, which may not be available in all scenarios.
- **Bandwidth and Latency:** Streaming video data for object detection may require high bandwidth, especially if using cloud-based solutions. Latency in

transmitting data for processing could hinder the real-time performance of the system.

- **Data Availability:** High-quality, diverse datasets are essential for training robust models. However, obtaining or annotating such datasets can be time-consuming and expensive.
- **Legal and Regulatory Constraints:** In many regions, surveillance and data collection are tightly regulated. Adhering to these laws is crucial when deploying object detection systems in public spaces.
- **Cost of Deployment:** Setting up a real-time detection system, especially on a large scale, can be costly. The hardware cameras, sensors and software licenses, computational resources required for real-time detection may be expensive for smaller scale projects.

Chapter 6

6. Conclusion:

In this project, we developed an object detection system using the YOLOv8 model, a CNN based approach optimized for real-time detection. The model was successfully implemented for both image-based object detection providing accurate results in real-time scenarios. This demonstrates the powerful capabilities of YOLOv8 in detecting various objects, making it suitable for a range of applications, from surveillance to autonomous systems.

6.1 Limitations:

- **Accuracy in Complex Scenarios:** While YOLOv8 performs well in many scenarios, its accuracy may decrease when detecting objects in highly cluttered or occluded environments. Complex backgrounds and overlapping objects may lead to false positives or missed detections.
- **Model Size and Inference Time:** The size of the pre-trained YOLOv8 model may pose challenges for systems with limited computational resources, such as edge devices or mobile platforms. While YOLOv8 is optimized for real-time performance, inference times may increase with larger, more complex models or when handling high-resolution video feeds.
- **Limited Generalization:** The model is trained on specific datasets COCO dataset and its performance on other unseen datasets or uncommon objects may be suboptimal. Fine-tuning on a custom dataset is required for better performance in such cases.

6.2 Future Works

- **Model Optimization for Edge Devices:** One of the primary areas of improvement is to optimize YOLOv8 for deployment on edge devices, such as mobile phones or embedded systems. This could involve reducing the model size, improving inference speeds, and using hardware acceleration techniques using GPUs or specialized AI chips.
- **Integration with Other Systems:** In future versions, the object detection model could be integrated with other systems, such as automated surveillance systems, smart home devices, or autonomous vehicles. This integration would allow for real-time decision making and actions based on detected objects, such as triggering alarms, sending notifications, or controlling robotic systems.
- **Multi-object Tracking and Behavior Analysis:** Adding multi-object tracking capabilities could allow the system to not only detect objects but

also track their movement across frames. Combining this with behavior analysis could enable use cases such as monitoring suspicious activities or recognizing patterns in crowd behavior.

- **Improved Detection in Dynamic Environments:** Future work could focus on enhancing the model's ability to handle dynamic and challenging environments, including varying lighting conditions, motion blur, and extreme weather. Developing adaptive models or combining YOLOv8 with other algorithms could address these challenges.

Appendix:

Preprocessing:

```
import os
import cv2
import numpy as np
from pycocotools.coco import COCO
from tqdm import tqdm
import albumentations as A

# Paths for COCO dataset
data_dir = "/kaggle/working"
train_images_dir = os.path.join(data_dir, "/kaggle/working/path/to/output/trainimages")
train_ann_file=os.path.join(data_dir,"/kaggle/working/path/to/output/trainannotations_balanced.json")

# Load COCO dataset
coco = COCO(train_ann_file)
img_ids = coco.getImgIds()

# Preprocessing config
IMAGE_SIZE = (512, 512)
AUGMENT = False # Set to True to enable augmentations

# Define augmentation pipeline using albumentations
transform = A.Compose([
    A.Resize(IMAGE_SIZE[0], IMAGE_SIZE[1]), # Resize to target size
    A.HorizontalFlip(p=0.5), # Horizontal flip with 50% probability
    A.RandomBrightnessContrast(p=0.2), # Random brightness and contrast
    A.Rotate(limit=10, p=0.5), # Random rotation between -10 and 10 degrees
])

# Preprocessing function
def preprocess_image(img_path, augment=False):
    img = cv2.imread(img_path)
    if img is None:
```

```

        return None

img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # Convert to RGB
if augment:
    augmented = transform(image=img)
    img = augmented["image"]
else:
    img = cv2.resize(img, (IMAGE_SIZE[0], IMAGE_SIZE[1]))
# Normalize to [0, 1]
img = img.astype(np.float32) / 255.0
return img

# Function to get category label (class ID) for each image
def get_main_label(img_id):
    ann_ids = coco.getAnnIds(imgIds=[img_id])
    anns = coco.loadAnns(ann_ids)
    if anns:
        return anns[0]['category_id'] # Get the first category_id (for simplicity)
    return -1 # Return -1 if no valid annotation exists

# Preprocess all training images
X = []
y = []
print("Preprocessing COCO training images...")
for img_id in tqdm(img_ids[:1000]): # Process a subset (first 1000) for testing purposes
    img_info = coco.loadImgs(img_id)[0]
    img_path = os.path.join(train_images_dir, img_info['file_name'])
    img = preprocess_image(img_path, augment=AUGMENT)
    if img is None:
        continue
    label = get_main_label(img_id)
    if label == -1: # Skip if no valid label

```



```
        continue
    X.append(img)
    y.append(label)
# Convert lists to numpy arrays for further use (training)
X = np.array(X)
y = np.array(y)
print(f" Preprocessed {len(X)} images. Shape: {X.shape}, Labels: {y.shape}")
np.save("X_train.npy", X)
np.save("y_train.npy", y)
```

References

- [1] Vijiyakumar, K., Govindasamy, V. and Akila, V., 2024. An effective object detection and tracking using automated image annotation with inception based faster R-CNN model. *International Journal of Cognitive Computing in Engineering*, 5, pp.343-356.
- [2] Vaishnavi, K., Reddy, G.P., Reddy, T.B., Iyengar, N. and Shaik, S., 2023. Real-time object detection using deep learning. *Journal of Advances in Mathematics and Computer Science*, 38(8), pp.24-32.
- [3] Talib, M., Al-Noori, A.H. and Suad, J., 2024. YOLOv8-CAB: Improved YOLOv8 for Real-time object detection. *Karbala International Journal of Modern Science*, 10(1), p.5.
- [4] Rani, G.H.S., Suvarna, D., Ammani, P.T., Jeetesh, E. and Chandu, K.S., REAL TIME OBJECT DETECTION USING MOBILENET-SSD WITH OPENCV.
- [5] Wang, A., Chen, H., Liu, L., Chen, K., Lin, Z., Han, J. and Ding, G., 2024. YOLOv10: Real-time end-to-end object detection. *arXiv preprint arXiv:2405.14458*.
- [6] Reis, D., Kupec, J., Hong, J. and Daoudi, A., 2023. Real-time flying object detection with YOLOv8. *arXiv preprint arXiv:2305.09972*.
- [7] Chen, Y., Yuan, X., Wu, R., Wang, J., Hou, Q. and Cheng, M.M., 2023. YOLO-ms: rethinking multi-scale representation learning for real-time object detection. *arXiv preprint arXiv:2308.05480*
- [8] Cheng, T., Song, L., Ge, Y., Liu, W., Wang, X. and Shan, Y., 2024. YOLO-world: Real-time open-vocabulary object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 16901-16911).
- [9] Liu, S., Zha, J., Sun, J., Li, Z. and Wang, G., 2023, July. EdgeYOLO: an edge-real-time object detector. In *2023 42nd Chinese Control Conference (CCC)* (pp. 7507-7512). IEEE.
- [10] Saha U, Ahamed IU, Hossain MI. YOLOv8 for bangla license plate recognition: Advancing realtime object detection in localized contexts. In *2024 7th International Conference on Informatics and Computational Sciences (ICICoS) 2024 Jul 17* (pp. 413-418). IEEE.
- [11] Afdhal A, Saddami K, Arief M, Sugiarto S, Fuadi Z, Nasaruddin N. MXT-YOLOv7t: An Efficient Real-Time Object Detection for Autonomous Driving in Mixed Traffic Environments. *IEEE Access*. 2024 Nov 27.
- [12].Reis, D., Kupec, J., Hong, J. and Daoudi, A., 2023. Real-time flying object detection with YOLOv8. *arXiv preprint arXiv:2305.09972*.

- [13] Yu, G., Chang, Q., Lv, W., Xu, C., Cui, C., Ji, W., Dang, Q., Deng, K., Wang, G., Du, Y. and Lai, B., 2021. PP-PicoDet: A better real-time object detector on mobile devices. arXiv preprint arXiv:2111.00902.
- [14] Upulie, H.D.I. and Kuganandamurthy, L., 2021. Real-time object detection using YOLO: a review. Real-Time Object Detection using YOLO: A review.
- [15] Wang, C.Y., Bochkovskiy, A. and Liao, H.Y.M., 2023. YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 7464-7475).
- [16] Zhou, H., Kong, M., Yuan, H., Pan, Y., Wang, X., Chen, R., Lu, W., Wang, R. and Yang, Q., 2024. Real-time underwater object detection technology for complex underwater environments based on deep learning. *Ecological Informatics*, 82, p.102680.
- [17] Vaithianathan, M., 2024. Real-Time Object Detection and Recognition in FPGA-Based Autonomous Driving Systems. *International Journal of Computer Trends and Technology*, 72(4), pp.145-152.
- [18] Guemas, E., Routier, B., Ghelfenstein-Ferreira, T., Cordier, C., Hartuis, S., Marion, B., Bertout, S., Varlet-Marie, E., Costa, D. and Pasquier, G., 2024. Automatic patient-level recognition of four Plasmodium species on thin blood smear by a real-time detection transformer (RT-DETR) object detection algorithm: a proof-of-concept and evaluation. *Microbiology Spectrum*, 12(2), pp.e01440-23.
- [19] Dubey, S.K., Satyanarayana, J.V. and Mohan, C.K., 2024, April. False positive elimination in object detection methods for videos. In *Sixteenth International Conference on Machine Vision (ICMV 2023)* (Vol. 13072, pp. 146-153). SPIE.